Lab 4: VibeCoding — Full Chat History

Target Shooting game — bug fix + feature implementation, exported for lab submission

NAME - ANIRUDHA ALAGAWADI

SRN – PES2UG24CS067

SECTION – 5B

CLAUDE CHAT HISTORY LINK - https://claude.ai/share/71ec2be2-d6e4-48fc-aaae-78935b763595

User

Uploaded Lab_4_VibeCoding_Student_handout.docx — the lab assignment: use an AI vibe-coding tool to fix a broken game repo and add features, individually, within 3-4 prompt attempts, then push to the student's own fork (not the main SETAPESU26 repo). Also shared the GitHub link:

https://github.com/SETAPESU26/04_target_shooting

Claude

Read the assignment and the repo's README. Identified the task from the README: a Pygame "Target Shooting" game with one deliberate bug (hit detection) and three features to add (moving targets, combo scoring, a 30-second timed round). Provided a workflow and four ready-to-use prompts:

- Fork the repo, clone it, record a 10-second "before" video.
- Prompt 1: fix the hit-detection bug (circular hit test instead of a shifted bounding rect).
- Prompt 2: add moving targets with two speed tiers and wall bouncing.
- Prompt 3: add combo scoring (points per hit scale with a streak multiplier; a miss resets it).
- Prompt 4: add a 30-second countdown round with a restart.

Also gave a test checklist (edge-accuracy clicks, bounce behavior, combo behavior, timer behavior) and reminded the student not to open a PR against the SETAPESU26 repo.

User

Asked for the git fork command.

Claude

Explained forking isn't a plain git command — it happens on GitHub itself. Gave two routes:

```
# Option 1: GitHub website
#  Click 'Fork' on the repo page, then 'Create fork'
git clone https://github.com/YOUR_USERNAME/04_target_shooting.git
cd 04_target_shooting

# Option 2: GitHub CLI
gh repo fork SETAPESU26/04_target_shooting --clone
```

Also showed how to verify remotes (git remote -v) and push safely to the fork (origin), not upstream.

User

Uploaded the actual repo contents as 04_target_shooting-main.zip.

Claude

Unzipped and inspected the real source: main.py, game/target.py, game/hit_detection.py, game/game_engine.py, game/renderer.py. Confirmed the bug matched the README description exactly: Target.get_bounding_rect() builds a pygame.Rect from (x, y) treating it as a top-left corner, when (x, y) is actually the circle's center — shifting the hitbox down-right of the drawn circle.

Task 1 fix — game/hit_detection.py — replaced the bounding-rect test with a true circular distance test:

```
def check_hit(targets, click_pos):
    click_x, click_y = click_pos
    for target in targets:
        dx = click_x - target.x
        dy = click_y - target.y
        distance_sq = dx * dx + dy * dy
        if distance_sq <= target.radius * target.radius:
            return target
    return None
```

Task 2 fix — game/target.py — added velocity fields and an update() method that moves the target and bounces it off the play-area edges, correcting for radius so the whole circle stays in bounds:

```
def __init__(self, x, y, radius=28, color=(230, 90, 70), vx=0.0, vy=0.0):
    ...
    self.vx = vx
    self.vy = vy

def update(self, dt, width, height):
    self.x += self.vx * dt
    self.y += self.vy * dt
    if self.x - self.radius < 0:
        self.x = self.radius; self.vx = -self.vx
    elif self.x + self.radius > width:
        self.x = width - self.radius; self.vx = -self.vx
    # (same pattern for y / height)
```

Task 3 & 4 — game/game_engine.py — rewritten to add:

- Two speed tiers (SPEED_TIERS = [80, 170] px/s) assigned randomly at spawn with a random direction angle.
- Score and combo: each hit adds 10 × combo, combo increments up to a cap of 5, any miss resets combo to 1.
- A 30-second round timer (ROUND_DURATION = 30.0) that freezes input at zero and shows the final score, with a restart() method to reset everything.

main.py — updated to compute real per-frame delta time from clock.tick(60) and pass it into engine.update(dt); added a KEYDOWN handler for the R key to call engine.restart().

Ran a headless smoke test (SDL_VIDEODRIVER=dummy) simulating frames, a hit, a miss, and forcing the timer to zero — confirmed score/combo/game-over logic worked with no exceptions. Packaged the fixed project as target-shooting-fixed.zip and presented it, with instructions to drop it into the student's fork, record the "after" video, and push.

User

Asked again for the git fork command (same as before).

Claude

Repeated the fork instructions (GitHub website Fork button, or gh repo fork), plus the push commands for pushing to the student's own fork rather than SETAPESU26's repo.

User

Uploaded an excerpt referencing 'pip install -r requirements.txt' and asked how to initialize a venv.

Claude

Walked through creating, activating, and using a virtual environment:

```
python3 -m venv venv
source venv/bin/activate        # Mac/Linux
venv\Scripts\activate.bat        # Windows cmd
pip install -r requirements.txt
python main.py
deactivate
```

Confirmed the repo's existing .gitignore already excluded venv/, so no extra step was needed there.

User

Ran the install from the wrong directory and got: "ERROR: Could not open requirements file: No such file or directory: 'requirements.txt'".

Claude

Identified that requirements.txt lives inside the target-shooting/ subfolder, not the repo root, and gave the corrected commands (cd target-shooting first).

User

Ran the install inside the correct folder; pygame failed to build from source with errors about missing sdl2-config, freetype2, and eventually "RuntimeError: Unable to run 'sdl-config'", ending in ModuleNotFoundError: No module named 'pygame'.

Claude

Diagnosed the root cause: the venv was running Python 3.14, for which classic pygame has no prebuilt wheel yet, so pip fell back to compiling from source — which requires SDL/freetype development headers that weren't installed. Gave two options:

- Option A (recommended): rebuild the venv with an older Python (3.10–3.12) that has prebuilt pygame wheels.
- Option B: keep Python 3.14 and install the SDL/freetype build dependencies via apt so pygame can compile.

User

Checked available Python versions: only python3.14 was installed.

Claude

Suggested installing python3.12 via apt, with a fallback to the deadsnakes PPA if not in the default repos, plus Option B (SDL build deps) as an alternative that avoids changing Python versions.

User

Ran sudo apt update && sudo apt install python3.12 python3.12-venv — package not found in repositories.

Claude

Recognized the system was on a very new Ubuntu release ("resolute", i.e. 25.10) whose default repos only ship Python 3.14, and where deadsnakes likely hadn't published packages yet. Searched the web and found that pygame-ce (the actively maintained community fork of pygame) added Python 3.14 wheel support. Recommended trying pygame-ce first as a fast, drop-in fix, with the SDL build-dependency route as a fallback.

```
pip uninstall pygame -y
pip install pygame-ce
python main.py
```

Also updated requirements.txt in the fixed project to pygame-ce>=2.5.0.

User

Confirmed pygame-ce installed cleanly and the game ran.

Claude

Gave a full verification checklist covering all four tasks (edge-accurate hit detection, two-speed bouncing movement, climbing/resetting combo, 30-second timer with restart on R), and reminded the student to update requirements.txt in their actual repo to pygame-ce.

Step-by-step debugging session

User

Asked to debug the code step by step with prompts.

Claude

Proposed testing Task 1 first: click just outside vs. just inside a target's edge on all four sides, expecting symmetric hit/miss behavior (unlike the original shifted-hitbox bug).

User

Confirmed: yes, it behaves correctly on all sides.

Claude

Moved to Task 2 verification: asked about (1) two distinct speeds, (2) clean wall bouncing, (3) accurate hits while moving.

User

Reported: distinct speeds — yes; bouncing — "doesn't block cleanly, just appears at the final position"; edge hit registration while moving — yes.

Claude

Asked targeted diagnostic questions to isolate whether this was a frame-rate/performance issue or a logic bug in the bounce code — specifically whether the jump happened only at walls or elsewhere, and what "appears at final position" looked like (teleport to the opposite side vs. snapping at the edge).

User

Clarified: not laggy at all — it visibly teleports to the opposite side of the screen.

Claude

Asked further isolating questions: does it teleport only at edges or also mid-screen; does it happen to every target or specific ones; mirror position vs. random spot; and crucially, whether it correlates with clicking.

User

Answered: happens anywhere on screen (1), to any target (2), to a random new spot (3), and specifically right when clicking (4).

Claude

Identified this as expected behavior, not a bug: per the README's original design, a hit removes the clicked target and spawns a brand-new one at a random location — which looks exactly like a "teleport" but is actually a different target being created. Asked the student to isolate pure bounce behavior by watching one target with zero clicking.

User

Reported: with no clicking, the targets don't move at all.

Claude

Recognized this as a strong signal that the movement code might simply be missing from the student's actual local files, and asked to see the real local game/target.py and game/game_engine.py contents to confirm.

User

Confirmed: still using the freshly forked repo, not the fixed files that had been provided earlier.

Claude

Explained the whole mystery was explained by this: the student was testing the original, unmodified repo, which has no movement/combo/timer code at all. Offered two paths: (A) copy in the previously generated fixed files directly, or (B) re-run the actual prompt-and-fix cycle against the student's real files for a more authentic chat-history record.

User

Copied the fixed files in; confirmed all bugs are now fixed.

Claude

Recapped that all four tasks were now verified against the real repo (hit detection, movement, and — pending an explicit re-check — combo and timer), and gave next steps: record before/after videos, export the chat, and push.

Git commit and push troubleshooting

User

Ran git checkout -b my-branch, git add ., git commit, and git push — push failed with "no upstream branch", and git status showed an unexpected staged file named '=2.5.0'.

Claude

Diagnosed the stray '=2.5.0' file as shell redirection: an unquoted ">=2.5.0" in an earlier command had been interpreted by bash as "redirect output into a file called 2.5.0", producing junk. Recommended removing it, verifying requirements.txt content, and amending the commit before pushing with --set-upstream.

User

Removed the stray file; requirements.txt still showed the old pygame>=2.5.0 line.

Claude

Flagged that requirements.txt needed correcting to pygame-ce>=2.5.0, and separately asked the student to check for a possible nested target-shooting/target-shooting/ folder given the path shown in the git rm output.

User

Confirmed via ls and find that there was no duplicate folder.

Claude

Clarified that git rm paths are always shown relative to the repo root, so the earlier output was expected and not a sign of duplication. Gave the final corrected sequence:

```
echo "pygame-ce>=2.5.0" > requirements.txt
```

```
git add requirements.txt
git commit --amend -m "Fix hit detection bug; add moving targets, combo scoring, timed
round"
git push --set-upstream origin my-branch
```

User

Ran the commit/push sequence successfully — branch my-branch pushed to the student's fork (Aanirudhh/04_target_shooting).

Claude

Confirmed the push succeeded cleanly, and warned not to click GitHub's suggested "Create a pull request" link unless the base repo was verified to be the student's own fork — not SETAPESU26/04_target_shooting. Outlined remaining submission steps: record before/after videos, export the chat, organize a Lab-4 folder, and optionally merge my-branch into main on the fork.

User

Asked to confirm: "Do not raise PR to the main repo (SETAPESU26) — we are not doing this right?"

Claude

Confirmed compliance: the push went to the student's own fork (Aanirudhh/04_target_shooting), not the original SETAPESU26 repo. Explained that GitHub's PR suggestion link only proposes a PR within the student's own fork (merging my-branch into that fork's main), which is unrelated to and does not violate the rule against opening a PR against the original assignment repo. Clarified that no PR is actually required at all, since the work was already pushed directly to the fork.

Export request

User

Shared a claude.ai share link and asked for the chat to be converted into a Word document.

Claude

The share link could not be fetched directly (client-rendered page), so this document was generated directly from the full conversation history already available in context.